SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

OPTIMALIZÁCIA ROZMEROV VSTUPNÝCH SNÍMOK OBLOHY PRE ODHAD SLNEČNÉHO ŽIARENIA

SEMESTRÁLNA PRÁCA

Študijný program:	Multimediálne informačné a komunikačné technológie
Predmet:	I-MKR – Metódy klasifikácie a rozhodovania
Prednášajúci:	doc. Ing. Juraj Kačur, PhD.
Cvičiaci:	doc. Ing. Juraj Kačur, PhD.

Bratislava 2026 **Bc. Lukáš Patrnčiak, Bc. Andrej Tomčík, Bc. Juraj Ševčík**

SÚHRN

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Študijný program:	Multimediálne informačné a komunikačné technológie
Autor:	Bc. Lukáš Patrnčiak, Bc. Andrej Tomčík, Bc. Juraj Ševčík
Semestrálna práca:	Optimalizácia rozmerov vstupných snímok oblohy pre odhad slnečného žiarenia
Vedúci práce:	doc. Ing. Juraj Kačur, PhD.
Miesto a rok predloženia práce:	Bratislava 2026

Táto práca sa zaoberá návrhom a optimalizáciou konvolučných neurónových sietí (CNN) pre odhad intenzity slnečného žiarenia na základe obrazových dát oblohy. Hlavným cieľom je analyzovať vplyv rozmerov vstupných obrázkov na presnosť regresného modelu a identifikovať optimálnu konfiguráciu vstupných dát pre rôzne architektúry vlastných CNN modelov.

V rámci práce boli navrhnuté tri architektúry konvolučných neurónových sietí s rôznou zložitosťou. Pre každú architektúru boli systematicky testované rôzne rozmery vstupných obrázkov spolu s kombináciami hyperparametrov, ako sú rýchlosť učenia, veľkosť spracovanej dávky (batchu), dropout a počet neurónov v plne prepojenej vrstve.

Modely boli trénované a vyhodnocované pomocou metrík strednej absolútnej chyby (MAE), strednej kvadratickej chyby (MSE) a odmocniny zo strednej kvadratickej chyby (RMSE). Súčasne bol analyzovaný čas trénovania a počet potrebných epoch, čím sa zohľadnila aj výpočtová náročnosť jednotlivých konfigurácií.

Výsledky ukazujú, že vhodná voľba rozmeru vstupných obrázkov má významný vplyv na presnosť modelu aj jeho výpočtovú efektívnosť. Na základe experimentov boli identifikované optimálne rozmery vstupov pre jednotlivé architektúry CNN, ktoré dosahujú najnižšiu chybu odhadu slnečného žiarenia.

Kľúčové slová: konvolučné neurónové siete, odhad slnečného žiarenia, snímky oblohy, rozlíšenie obrazu, optimalizácia rozmerov, hlboké učenie, predikcia solárneho žiarenia

Obsah

Úvod **1**

1 Použité metódy **2**

1.1 Konvolučné neurónové siete 2

1.2 Metriky 3

2 Návrh riešenia **5**

2.1 Štruktúra dát 5

2.2 Navrhnuté architektúry 6

2.3 Experimenty 8

3 Implementácia **11**

3.1 predspracovanie dát 12

3.2 Implementcia architektúr CNN 12

3.3 Automatizácia experimentov 13

3.4 Vyhodnotenie modelov 13

3.5 Použitý HW 14

4 Výsledky **15**

Záver **18**

Zoznam obrázkov a tabuliek

Tabuľka 2 Prehľad navrhnutých architektúr CNN 8
Tabuľka 3 tabuĺka: Tabuľka závyslostí archytektúry Small 16
Tabuľka 4 tabuĺka: Tabuľka závyslostí archytektúry Medium 16
Tabuľka 5 tabuĺka: Tabuľka závyslostí archytektúry Large 17

Zoznam skratiek

CNN konvolučná neurónová sieť

MAE stredná absolútna chyba

MSE stredná kvadratická chyba

ReLU Rectified Linear Unit

RMSE koreňová stredná kvadratická chyba

Úvod

V tejto práci sa zameriavame na problém odhadu intenzity slnečného žiarenia z fotografií oblohy. Pri návrhu modelov založených na konvolučných neurónových sieťach zohráva významnú úlohu nielen architektúra siete a jej hyperparametre, ale aj vlastnosti vstupných dát. Jedným z kľúčových faktorov je rozmer vstupného obrázka, ktorý ovplyvňuje presnosť modelu a aj výpočtovú náročnosť trénovania.

Cieľom tejto práce je analyzovať vplyv rozmerov vstupných obrázkov na kvalitu regresného modelu a nájsť optimálne nastavenie pre rôzne architektúry konvolučných neurónových sietí. V rámci experimentov sú systematicky testované viaceré konfigurácie modelov a hyperparametrov, pričom výsledky sú vyhodnocované pomocou štandardných chybových metrík a časovej náročnosti trénovania.

Výsledkom práce je identifikácia optimálnych kombinácií architektúry a veľkosti vstupných obrázkov, ktoré poskytujú najlepší kompromis medzi presnosťou odhadu a výpočtovou efektívnosťou, ktorá zohrávala nemalú úlohu pri návrhu celej implementácie vzhľadom na veľkosť datasetu.

1 Použité metódy

V tejto kapitole budú popísané jednotlivé metódy použité v rámci implementácie a vyhodnotenia optimálnych rozmerov obrázkov [**deepLearning**, **globalPooling**, **dropout**, **batchNormalization**].

1.1 Konvolučné neurónové siete

Konvolučné neurónové siete (CNN) predstavujú špecializovaný typ neurónových sietí určený na spracovanie dát s mriežkovou štruktúrou, ako sú napríklad obrázky. Charakteristickou vlastnosťou týchto sietí je využitie operácie konvolúcie, ktorá umožňuje efektívnu extrakciu príznakov zo vstupných dát. Typická architektúra konvolučnej neurónovej siete sa skladá z viacerých základných vrstiev:

- **Vstupná vrstva:** slúži na načítanie vstupných dát, napríklad obrazov definovaného rozmeru. V prípade konvolučných neurónových sietí býva vstup reprezentovaný trojrozmerným tenzorom obsahujúcim výšku, šírku a farebné kanály obrazu.

- **Konvolučné vrstvy:** realizujú extrakciu príznakov zo vstupných dát pomocou konvolučných jadier (filtrov). Príznaky predstavujú charakteristické vlastnosti obrazu, napríklad hrany, textúry, tvary alebo jasové prechody, ktoré neurónová sieť automaticky extrahuje a využíva pri predikcii výslednej hodnoty. Tieto vrstvy umožňujú detekciu hrán, textúr a ďalších charakteristických vlastností obrazu. Počas tréningu sa hodnoty filtrov automaticky optimalizujú tak, aby model dokázal čo najlepšie reprezentovať vstupné dáta.

- **Aktivačné funkcie:** zavádzajú do modelu nelinearitu, vďaka ktorej je možné aproximovať zložité vzťahy medzi vstupmi a výstupmi modelu. Medzi najpoužívanejšie patrí funkcia ReLU (*Rectified Linear Unit*), ktorá nastavuje záporné hodnoty na nulu a kladné hodnoty ponecháva nezmenené. Definícia tejto funkcie vyzerá nasledovne:

$$ReLU(x) = \max(0, x)$$

- **Normalizačné vrstvy:** zabezpečujú normalizáciu aktivácií jednotlivých vrstiev počas tréningu, čím prispievajú k stabilnejšiemu a rýchlejšiemu učeniu modelu. Často sa využíva technika Batch Normalization, ktorá znižuje citlivosť modelu na počiatočné nastavenie parametrov.

- **Pooling vrstvy:** slúžia na redukciu priestorových rozmerov príznakových máp a zníženie výpočtovej náročnosti siete. Najčastejšie sa používa max pooling, pri ktorom sa z vybraného regiónu zachováva iba maximálna hodnota aktivácie.
- **Global pooling vrstvy:** prevádzajú dvojrozmerné príznakové mapy na jednorozmerný vektor pomocou výpočtu globálnych štatistík, napríklad priemeru. Použitie global pooling vrstiev znižuje počet parametrov modelu a tým aj riziko pretrénovania.
- **Plne prepojené vrstvy (Dense):** realizujú kombináciu extrahovaných príznakov a vykonávajú finálne rozhodovanie modelu. Tieto vrstvy sú tvorené klasickými perceptrónmi, pričom každý neurón je prepojený so všetkými neurónmi predchádzajúcej vrstvy. Slúžia na vytváranie vzťahov medzi príznakmi extrahovanými v konvolučných vrstvách a na vytvorenie finálnej predikcie modelu.
- **Regularizačné vrstvy (Dropout):** slúžia na zníženie rizika pretrénovania modelu náhodnou deaktiváciou časti neurónov počas tréningu. Tým sa zabraňuje prílišnému prispôsobeniu modelu trénovacím dátam.
- **Výstupná vrstva:** poskytuje finálny výstup neurónovej siete vo forme klasifikácie alebo numerickej predikcie. Typ aktivačnej funkcie výstupnej vrstvy závisí od charakteru riešenej úlohy.

1.2 Metriky

V tejto kapitole budú popísané metriky, ktoré boli použité na vyhodnotenie úspešnosti navrhnutých modelov konvolučných neurónových sietí. Vzhľadom na to, že cieľom modelu je predikcia spojitej numerickej hodnoty intenzity slnečného žiarenia (*Irradiance*), riešená úloha predstavuje regresný problém. Výstupom modelu teda nie je diskrétna trieda, ako je tomu pri klasifikačných úlohách, ale numerická hodnota reprezentujúca odhadovanú intenzitu žiarenia.

Ako stratová funkcia počas tréningu bola použitá metrika MSE (*Mean Squared Error*), ktorá vyjadruje priemernú kvadratickú chybu medzi skutočnými a predikovanými hodnotami. Ako doplnková metrika počas tréningu bola sledovaná metrika MAE (*Mean Absolute Error*), ktorá predstavuje priemernú absolútnu odchýlku predikcie od skutočnej hodnoty. Na rozdiel od MSE nepriraďuje veľkým chybám výrazne vyššiu váhu, a preto poskytuje jednoduchšiu a ľahšie interpretovateľnú informáciu o priemernej veľkosti chyby modelu.

Po natrénovaní modelu boli výsledky vyhodnotené aj v pôvodnej mierke cieľovej premennej žiarenia (*Irradiance*). Okrem metrík MSE a MAE bola vypočítaná aj metrika RMSE (*Root Mean Squared Error*), ktorá predstavuje druhú odmocninu z hodnoty MSE. Výhodou tejto metriky je, že jej výsledná hodnota sa nachádza v rovnakých jednotkách ako predikovaná veličina *Irradiance*, vďaka čomu je chyba modelu jednoduchšie interpretovateľná z praktického hľadiska.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$RMSE = \sqrt{MSE}$$

V uvedených rovniciach označujú jednotlivé symboly:

- n počet vzoriek,
- y_i skutočnú hodnotu cieľovej premennej,
- $\hat{y}_i$ hodnotu predikovanú modelom pre i-tu vzorku.

2 Návrh riešenia

V tejto kapitole bude popísaný návrh riešenia, predovšetkým štruktúra poskytnutých dát, ktoré zahŕňajú tabuľkové (*CSV*) súbory a obrázky oblohy.

2.1 Štruktúra dát

Štruktúra poskytnutých dát vo forme obrázkov použitá v práci je rozdelená na tri základné množiny:

1. **trénovaciu množinu**: slúži na učenie neurónovej siete a aktualizáciu parametrov modelu počas tréningového procesu,
2. **validačnú množinu**: používa sa na priebežné vyhodnocovanie modelu počas tréningu a na výber najvhodnejšej konfigurácie modelu bez priameho učenia,
3. **testovaciu množinu**: slúži na záverečné vyhodnotenie natrénovaného modelu na dátach, ktoré neboli použité počas procesu učenia ani validácie.

Každá časť datasetu je uložená v samostatnom priečinku a obsahuje obrazové dáta spolu so súborom vo formáte CSV, v ktorom sú uložené príslušné hodnoty meranej veličiny. Dataset má nasledujúcu štruktúru:

```
dataset/
|-- train/
|   |-- images/
|   |-- meteo_data.csv
|
|-- val/
|   |-- images/
|   |-- meteo_data.csv
|
|-- test/
|   |-- images/
|   |-- meteo_data.csv
```

Priečinok **images** obsahuje vstupné obrazové dáta vo formáte PNG. Ku každému obrázku je v príslušnom CSV súbore uvedený jeho názov a hodnota cieľovej premennej. V tejto práci boli použité nasledovné stĺpce:

- **PictureName:** obsahuje názov obrazového súboru a slúži na priradenie jednotlivých záznamov k príslušným obrazovým dátam,
- **Irradiance:** obsahuje hodnotu slnečného žiarenia, ktorá predstavuje cieľovú hodnotu predikcie.

Tieto dva stĺpce boli zvolené z dôvodu, že cieľom práce je analyzovať vplyv rozmerov obrazových dát na presnosť predikcie intenzity slnečného žiarenia výhradne na základe obrazových vstupov bez využitia doplnkových meteorologických údajov.

Pri načítaní dát sa ku každému záznamu v CSV súbore priradí úplná cesta k príslušnému obrázku. Záznamy, pri ktorých chýba cieľová hodnota alebo príslušný obrazový súbor, sú z datasetu odstránené. Tým sa zabezpečí, že model pracuje iba s platnými dátami.

Trénovacia množina slúži na učenie modelu, validačná množina sa používa na sledovanie priebehu učenia a výber najlepšej konfigurácie modelu. Testovacia množina je použitá až na záverečné vyhodnotenie natrénovaného modelu na dátach, ktoré neboli použité počas tréningu.

2.2 Navrhnuté architektúry

V rámci práce boli navrhnuté a porovnané tri architektúry konvolučných neurónových sietí, ktoré sa líšia najmä počtom konvolučných vrstiev, počtom filtrov a celkovou zložitosťou modelu. Cieľom návrhu viacerých architektúr bolo overiť, ako vplýva veľkosť a hĺbka neurónovej siete na presnosť predikcie intenzity slnečného žiarenia pri rôznych rozmeroch vstupných obrázkov.

Vstupom do všetkých archytektúr boli RGB obrázky oblohy s rozmermi H x W x 3, kde H a W predstavujú výšku a šírku vstupných obrázkov ktorú sme v rámci testovania menili, použili sme velkosti 64x64, 96x96, 128x128, 160x160, 224x224 pixelov.

Navrhnuté architektúry boli označené ako *small*, *medium* a *large*. Všetky modely boli určené na riešenie regresnej úlohy, preto ich výstupná vrstva obsahovala jeden neurón, ktorý reprezentoval predikovanú hodnotu cieľovej premennej *Irradiance*.

- **Small CNN:** predstavuje najjednoduchšiu navrhnutú architektúru. Obsahuje dve konvolučné vrstvy s 16 a 32 filtrami. Po každej konvolučnej časti nasleduje normalizácia dávky a zmenšenie rozmerov pomocou max pooling vrstvy. Táto architektúra bola navrhnutá ako výpočtovo menej náročný model.
- **Medium CNN:** predstavuje stredne zložitú architektúru. Obsahuje tri konvolučné vrstvy s 32, 64 a 128 filtrami. V porovnaní s menšou architektúrou obsahuje viac

vrstiev a vyšší počet filtrov, vďaka čomu dokáže extrahovať zložitejšie obrazové príznaky. Súčasťou tejto architektúry sú aj dropout vrstvy, ktoré slúžia na zníženie rizika pretrénovania.

- **Large CNN:** predstavuje najzložitejšiu navrhnutú architektúru. Obsahuje viacero po sebe nasledujúcich konvolučných vrstiev s 32, 64, 128 a 256 filtrami. Táto architektúra umožňuje extrakciu väčšieho množstva príznakov z obrazových dát, avšak zároveň má vyššiu výpočtovú náročnosť.

Prehľad architektúr týchto navrhnutých CNN je možné vidieť v tabuľke 2. Spoločným prvkom všetkých navrhnutých architektúr bolo použitie aktivačnej funkcie ReLU, normalizačných vrstiev Batch Normalization, pooling vrstiev a finálnej vrstvy Global Average Pooling. Hlavnímy výhodou aktovačnej funkcie ReLU, ktorá patrí medzi najčastejšie používanné aktivačné funkcie, je jej jednoduchá implementácia a výpočtová nenárčnosť.

$$ReLU : f(x) = max(0, x)$$

Po extrakcii príznakov nasledovala plne prepojená vrstva, dropout vrstva a výstupná vrstva s jedným neurónom. Všetky navrhnuté architektúry využívajú jednotlivé vrstvy v nasledovnom poradí:

- **Conv2D vrstva:** prvá konvolučná vrstva slúži na extrakciu základných príznakov z obrazových dát, ako sú hrany, textúry alebo jednoduché tvary. Konvolučné filtre sa počas tréningu automaticky prispôsobujú charakteru vstupných dát. Vo všetkých konvolučných vrstvách bola použitá velkosť filtra 3x3.

- **Batch Normalization vrstva:** za konvolučnou vrstvou nasleduje normalizačná vrstva Batch Normalization, ktorá zabezpečuje normalizáciu aktivácií jednotlivých vrstiev počas tréningu. Použitie tejto vrstvy prispieva k stabilnejšiemu a rýchlejšiemu učeniu modelu a znižuje citlivosť na inicializáciu parametrov.

- **Druhá Conv2D vrstva:** druhá konvolučná vrstva umožňuje modelu extrahovať zložitejšie príznaky ešte pred redukciou priestorových rozmerov obrazu. Kombinácia viacerých konvolučných vrstiev za sebou zvyšuje schopnosť modelu zachytiť zložitejšie obrazové charakteristiky.

- **MaxPooling2D vrstva:** pooling vrstva využíva pooling okno s rozmrom 2x2 a slúži na zmenšenie priestorových rozmerov príznakových máp. Tým dochádza k zníženiu výpočtovej náročnosti modelu a zároveň k zachovaniu najvýznamnejších príznakov obrazu.

- **Dropout vrstva:** po pooling vrstve nasledovala regularizačná vrstva Dropout, ktorá počas tréningu náhodne deaktivuje časť neurónov. Cieľom tejto vrstvy je zníženie rizika pretrénovania modelu a zlepšenie generalizačných schopností neurónovej siete.

Takto navrhnutý blok vrstiev bol v architektúrach opakovane použitý s postupne rastúcim počtom konvolučných filtrov. Zvyšovanie počtu filtrov v hlbších vrstvách umožňuje modelu extrahovať zložitejšie a abstraktnejšie príznaky z obrazových dát.

Tabuľka 2: Prehľad navrhnutých architektúr CNN

Architektúra	**Konvolučné filtre**	**Pooling**	**Výstup**
Small CNN	16, 32	MaxPooling, GlobalAveragePooling	1 neurón
Medium CNN	32, 64, 128	MaxPooling, GlobalAveragePooling	1 neurón
Large CNN	32, 32, 64, 64, 128, 128, 256	MaxPooling, GlobalAveragePooling	1 neurón

Pri všetkých architektúrach bola použitá stratová funkcia MSE a optimalizačný algoritmus Adam. Tento algoritmus slúži na postupnú aktualizáciu parametrov neurónovej siete počas tréningu tak, aby sa minimalizovala chyba modelu. Tento algoritmus bol zvolený z dôvodu jeho stabilného a rýchleho učenia, pričom patrí medzi najčastejšie používané optimalizačné algoritmy pri trénovaní konvolučných neurónových sietí. [**adam**].

Počas tréningu bola sledovaná metrika MAE. Na zníženie rizika pretrénovania boli použité vrstvy Dropout a mechanizmus **Early Stopping**, ktorý ukončil tréning v prípade, že sa validačná chyba prestala zlepšovať.

Okrem toho bol použitý mechanizmus **PReduceLROnPlateau**, ktorý automaticky znižoval hodnotu učiaceho koeficientu v prípade, že sa validačná chyba počas viacerých epoch nezlepšovala. Cieľom tohto mechanizmu bolo umožniť modelu jemnejšie dolaďovanie parametrov a stabilnejšiu konvergenciu počas tréningu.

2.3 Experimenty

V rámci experimentálnej časti práce bolo realizované systematické porovnanie navrhnutých architektúr CNN pri rôznych rozmeroch vstupných obrázkov a rôznych nastaveniach hyperparametrov. Cieľom experimentov bolo analyzovať vplyv rozmerov obrazových dát na presnosť predikcie intenzity slnečného žiarenia a zároveň sledovať výpočtovú náročnosť jednotlivých nastavení modelov.

Experimenty boli realizované pre vyššie spomínané architektúry (2.2), pčitom pri experimentoch boli testované nasledovné rozmery vstupných obrázkov:

- 64×64,
- 96×96,
- 128×128,
- 160×160,
- 224×224.

Zvolené rozmery predstavujú často používané veľkosti vstupných obrazových dát pre CNN. Hodnoty boli vybrané tak, aby pokrývali menšie aj väčšie rozlíšenia obrazov a umožnili analyzovať vplyv rastúceho rozmeru vstupných dát na presnosť predikcie a výpočtovú náročnosť modelu. Menšie rozmery, ako napríklad 64×64, umožňujú rýchlejší tréning a nižšie hardvérové nároky, zatiaľ čo väčšie rozmery zachovávajú viac detailov v obraze. Rozmer 224×224 bol zvolený aj z dôvodu, že ide o štandardný vstupný rozmer používaný pri viacerých známych architektúrach CNN.

Maximálny počet tréningových epoch[1] bol nastavený na hodnotu 50. Okrem rozmerov vstupných dát boli experimentálne menené aj vybrané hyperparametre modelov:

- **rýchlosť učenia (learning rate):** táto hodnotaurčuje veľkosť kroku pri aktualizácii parametrov modelu počas tréningu. Vybrané hodnoty predstavujú často používané nastavenia pri trénovaní konvolučných neurónových sietí . Vyššia hodnota 0.001 umožňuje rýchlejšie učenie modelu, zatiaľ čo nižšia hodnota 0.0001 zabezpečuje stabilnejšie a jemnejšie dolaďovanie parametrov. Tieto hodnoty boli nastavené na:

- **dropout:** Nasledovne zvolené hodnoty predstavujú miernu regularizáciu modelu, ktorá umožňuje znížiť riziko pretrénovania bez výrazného obmedzenia schopnosti siete učiť sa z dát. Tieto hodnoty patria medzi často používané nastavenia pri CNN:
  - 0.2 =´20%,
  - 0.3 = 30%,

- **počet neurónov v Dense vrstve:** V práci boli zvolené hodnoty 64 a 128 neurónov s cieľom porovnať, či väčší počet neurónov umožní modelu lepšie zachytiť vzťahy v obrazových dátach a zlepšiť presnosť predikcie.

- **batch size:** Táto hodnota určuje počet vzoriek spracovaných modelom v jednom tréningovom kroku. V práci bola zvolená hodnota 64, ktorá predstavuje kompromis medzi rýchlosťou tréningu, pamäťovou náročnosťou a stabilným priebehom učenia modelu. To znamená, že model počas jedného tréningového kroku spracuje naraz 64

[1]Epocha je plné prejdenie celej trénovacej množiny modelom počas učenia. Tento proces zahŕňa postupné spracovanie všetkých obrázkov, výpočet chyby a aktualizáciu svojich parametrov.

obrazových dát. Napríklad pri trénovacej množine obsahujúcej 25 600 obrázkov je jedna epocha tvorená 400 tréningovými krokmi.

Počas tréningu bol použitý mechanizmus **Early Stopping**, ktorý automaticky ukončil tréning v prípade, že sa presnosť modelu na validačných dátach dlhší čas nezlepšovala. Tým sa zabránilo zbytočne dlhému tréningu a zároveň sa znížilo riziko pretrénovania modelu.

Súčasne bol použitý mechanizmus **ReduceLROnPlateau**, ktorý v prípade, že sa chyba modelu na validačných dátach prestala zlepšovať, automaticky znížil hodnotu učiaceho koeficientu (*learning rate*). Na začiatku tréningu sa model učí rýchlejšie pomocou väčších zmien parametrov, zatiaľ čo po znížení hodnoty learning rate dochádza k jemnejším úpravám modelu. Tento prístup umožňuje presnejšie dolaďovanie parametrov neurónovej siete a pomáha modelu nájsť vhodnejšie riešenie.

Každá konfigurácia modelu bola vyhodnotená pomocou metrík MSE, MAE a RMSE. Výsledky jednotlivých experimentov boli následne porovnané s cieľom identifikovať optimálny rozmer vstupných obrázkov pre jednotlivé architektúry konvolučných neurónových sietí.

3 Implementácia

Na implementáciu navrhnutého riešenia bol použitý programovací jazyk Python [**learningPython**] spolu s viacerými externými knižnicami určenými na spracovanie dát a ich vizualizáciu a tvorbu trénovanie konvolučných neurónových sietí. Implementácia bola realizovaná s rôrazom na jednoduché testovanie a automatizáciu experimentov rôznych konfigurácií a bola navrhnutá tak aby výsledky boli jednoducho replikovatelné. Pri implementácii boli použité nasledovné knižnice:

- **os** – práca so súborovým systémom a adresármi operačného systému,
- **math** – realizácia matematických operácií a výpočtov,
- **time** – meranie času a sledovanie trvania operácií,
- **random** – táto funkcia slúži na generovanie náhodných hodnôt ale pre zachovanie rovnakých podmienok pri trénovaný všetkých modelov a reprodukavetelnosť výsledkov je nastavená premenná RAND_ST na fixnú hodnotu čím sa zebezpečuje že náhodne generované čísla budu pri roznych behoch programu rovnaké.
- **pandas** – spracovanie a analýza tabuľkových dát,
- **numpy** – numerické výpočty a operácie nad viacrozmernými poľami,
- **matplotlib.pyplot** – vizualizácia výsledkov a tvorba grafov,
- **tensorflow** – framework určený na implementáciu a tréning neurónových sietí.

Implementácia sa skladala z viacerých časti ktoré slúžili na načítanie a kontrolu datasetu, predspracovanie vstupných dát, vytvorenie modelu, automatizáciu trénovania a vyhodnotenie výsledkov a generovanei grafov.

Na implementáciu konvolučnej neurónovej siete bolo využité rozhranie Keras, ktoré je súčasťou frameworku TensorFlow. Použitý bol modul **keras.Input** určený na definovanie vstupnej vrstvy neurónovej siete, modul **tensorflow.keras.layers** obsahujúci konvolučné, pooling a plne prepojené vrstvy a modul **tensorflow.keras.models** určený na vytváranie architektúr neurónových sietí.

Na optimalizáciu procesu učenia bol použitý algoritmus **keras.Input**, ktorý sa nachádza v module **tensorflow.keras.optimizers.Adam**. Súčasťou implementácie boli aj callback mechanizmy z modulu **tensorflow.keras.callbacks**, konkrétne:

- **EarlyStopping** – tento mechanizmus slúži na predčasné ukončenie tréningu modelu pri stagnácii validačnej chyby modelu, tento mechanizmus kontroluje zmenu vo validačnej chybe na konci každej trénovacej epochy. Jeho implemmntáciou sa znižuj riziko pretrénovania modelu a skracuje sa čas trénovania.
- **ModelCheckpoint** – toto je mechanizmus implementovaný na automatické uklada-

nie najlepšieho modelu počas tréningu, keď program dokončí trénovanie modelu a je prpravený trénovať ďalší model, tento mechanizmus ho automaticky uloži na zvolené miesto .

- **ReduceLROnPlateau** – táto funkcia zabezpečuje dynamickú úpravu rýchlosti učenia počas tréningu ak vyhodnotí zhoršenie konvergencie modelu.

Pre spracovanie vstupných obrázkov bol použitý modul **tensorflow.keras.preprocessing.image**, konkrétne funkcie:

- **load_img** – načítanie obrazových dát,
- **img_to_array** – prevod obrázkov do numerickej reprezentácie vo forme numerického poľa.

3.1 predspracovanie dát

Pred samotným tréningom modelov bolo potrebé vykonať predspracovania vstupných dát. Na načítanie a úpravu boli implementované funkcie **load_and_preprocess_image()** a **create_tensorflow_dataset()**. Každý vstupný obrázok bol najskôr načítaný z disku a následne transformovaný na požadovaný rozmer podľa aktuálne testovanej konfigurácie modelu. Po zmene rozmerov bol obrázok prevedený na numerické pole typu *NumPy array*. Hodnoty pixelov boli následne normalizované do intervalu $\langle 0, 1 \rangle$ vydelením hodnotou 255:

$$x_{norm} = \frac{x}{255}$$

Táto normalizácia prispieva k stabilnejšiemu priebehu učenia neurónovej siete a znižuje riziko numerických problémov počas optimalizácie modelu.

Okrem vstupných obrazových dát bola normalizovaná aj skúmaná hodnota *Irradiance*. A to pomocov implementovaním štandardizácie priemeru a smerodajnej odchýlky:

$$x_{norm} = \frac{x - \mu}{\sigma}$$

Kde μ je priemernú hodnotu cieľovej premennej a σ je smerodajná odchylka. Táto normalizácia zlepší stabilitu modelov, urýchli proces trénovania a zvýši rýchlosť konvergencie počas trénovania.

Po načítaní a normalizácii boli dáta prevedené do formátu **TensorFlow Dataset**, ktorý umožňuje rozdelovanie dát a používať (*batching*), miešanie dát (*shuffling*) a optimalizáciu načítavanie vstupných dát počas tréningu .

3.2 Implementcia architektúr CNN

Všetky chitektúry boli implementované cez sekvenčný model **tensorflow.keras.Sequential**. Každý model pozostáva z viacerých blokov konvolučných, normalizačných, batch, pooling,

dropout a regulačných vrstiev.

V konvolučnej vrstve sa využívali filtre o veľkosti 3x3 a aktivačná ReLU funkcia. Po každej skupine konvolučných vrstiev nasledovala pooling vrstva typu MaxPooling2D s veľkosťou pooling okna 2x2, ktorá znižuje priestorové rozmery príznakových máp. Na konci konvolučnej časti modelu je GlobalAveragePooling2D vrstva, na prevádzanie dvojrozmerných príznakových máp na jednorozmerný vektor. Táto vrstva tiež znižuje počet parametrov trénovaného modelu v porovnaní s klasickým flatten prístupom a toto tiež pomáha redukovať riziko overfittingu. Za global pooling vrstvou je dense vrstva s nastaviteľným počtom neurónov a následne dropout vrstva. Na výstupe je dense vrstva s jedným neurónom bez aktivačnej funkcie.

3.3 Automatizácia experimentov

Proces trénovanai modelov a ich vyhodnocovania bol v tejto implementácii automatizovaný a to tak že program postupne testoval rôzne kombinácie architektúr, veľkostí vstupných obrázkov a hyperparametrov. Pre každú kombináciu parametrov a archytektúry bol automaticky:

- vytvorený tréningový, validačný a testovací dataset,
- inicializovaný model neurónovej siete,
- natrénovaný model na dátach z trénovacieho datastu,
- uložené výsledné metriky aj s informáciamy o parametroch,
- uložený najlepší model,
- vygenerované grafy priebehu učenia a dosiahnutých výsledkov.

Počas tréningu sa zaznamenávali hodnoty tréningovej a validačnej chyby pre každú epochu. Tieto údaje boli následne použité na tvorbu grafov na zobrazenie dosiahnutých výsledkov a procesu trénovia. Výsledky všetkých experimentov sa ukladali do tabulky do výstupného formátu .csv, čo umožnuje ich následné porovnanie a analýzu pomocou korelácií medzi parametrami modelov a dosiahnutými výsledkami.

3.4 Vyhodnotenie modelov

Po dokončení tréningu boli jednotlivé modely vyhodnotené na testovacej množine kde pre každý model boli vypočítané MSE, MAE a RMSE a to v normalizovanej aj pôvodnej hodnote. Tiež bol na vyhodnotenie modelu boli zaznamenávané aj časi tréningu modelu a počet epoch potrebných na konvergenciu pre spusreeným funkcie earlystop a teda môžeme porovnať nielen presnosť rôznich testovaných modelov, ale aj ich výpočtovú náročnosť. Súčasťou implementácie bolo aj automatické generovanie grafov závislosti jednotlivých

hyperparametrov od výsledných metrík. Tieto grafy boli následne použité pri analýze vplyvu rozmerov vstupných obrázkov a hyperparametrov na presnosť modelov.

3.5 Použitý HW

Celý proces trénovania ako už bolo spomenuté je implementovaný v jednom python programe a je navrhnutý ako jeden celok ktorý od spustenia po skončenie nevyžaduje žiadny zásah a na konci automaticky poskytne grafy s dosiahnutími výsledkami. Na spustenie tohoto programu bol použitý software CuDDa (verzia) spôštaná cez WSL2 na GPU a CPU (doplniť čo sa použilo)

4 Výsledky

Proces testovania je automatizovaný a po dokončený všetkých modelov sú k dospozícii výsledky ich hodnotenia a grafy vyhodnocujúce vybrané sledované parametre.

Parametre ktoré sa o natrénovaných modeloch ukladaly:

- **training_time_seconds** – celkový čas potrebný na natrénovanie modelu v sekundách.
- **train_mae_norm / val_mae_norm / test_mae_norm** – (Mean Absolute Error) je priemerná absolútna chyba. Hovorí o tom, o koľko sa v priemere model mýli v normalizovaných jednotkách.
- **train_loss_norm / val_loss_norm / test_loss_norm** – snažíme sa minimalizovať, porovnanie straty napovedá či nedochádza k overfittingu, ak trénovacia straa klesá ale validačná začne stúpať
- **val_mse_original / test_mse_original** – (Mean Squared Error)
- **val_mae_original / test_mae_origina** – (Mean Absolute Error) – model sa v priemere mýli o X
- **val_rmse_original / test_rmse_original** – (Root Mean Squared Error)

Cielom je nájsť najnožšie test_mae_original a paramtre s ním súvisiace. Ak je RMSE oveľa vyššie ako MAE, model robí občasné, ale velmi veľké chyby train_loss_norm a val_loss_norm – ak velký rozdiel ide o overfitting – model zahod

Na vyhodnocovanie vplyvu jednotlivých parametrov na výsledok a čas trénovanie budeme používať koreláciu. Z tabulky vyplíva že jeniný parameter ktorý má významný

	čas trénovania	epoch trained	MSE	RMSE	MEA
velkosť obrázkov	0,5036	-0,0985	0,3508	0,3519	0,3802
learning rate	0,1644	0,1644	-0,2693	-0,2929	-0,3211
dropout rate	0,0503	0,0573	0,1383	0,1342	0,1420
danse units	0,0637	0,0841	-0,0939	-0,0813	-0,0831

vpliv na čas trénovania modelu je velksť vstupných obrázkov, tento parameter má tiež najväčší vpliv na všetky ostatné sledované paramete.

Z tabuliek môžeme vidieť že pri použití malej neurónovej siete má na presnosť natrénovaného modelu aj na čas trénovania največí vpliv velkosť vstupných obrázkov.

SMALL	čas trénovania	epoch trained	MSE	RMSE	MEA
velkosť obrázkov	0,6507	-0,1119	0,0634	0,1884	0,1884
learning rate	0,0977	-0,0344	-0,1501	-0,2491	-0,2491
dropout rate	-0,0525	-0,0306	0,0010	-0,0140	-0,0140
danse units	-0,0518	-0,0229	0,1054	0,0277	0,0277

Tabuľka 3: tabuĺka: Tabuľka závyslostí archytektúry Small

MEDIUM	čas trénovania	epoch trained	MSE	RMSE	MEA
velkosť obrázkov	0,8279	0,3614	0,3503	0,3777	0,4307
learning rate	0,1750	0,3246	-0,4008	-0,3976	-0,3800
dropout rate	-0,0910	-0,0710	0,2622	0,2932	0,2937
danse units	-0,0346	-0,0913	-0,1121	-0,1105	-0,0892

Tabuľka 4: tabuĺka: Tabuľka závyslostí archytektúry Medium

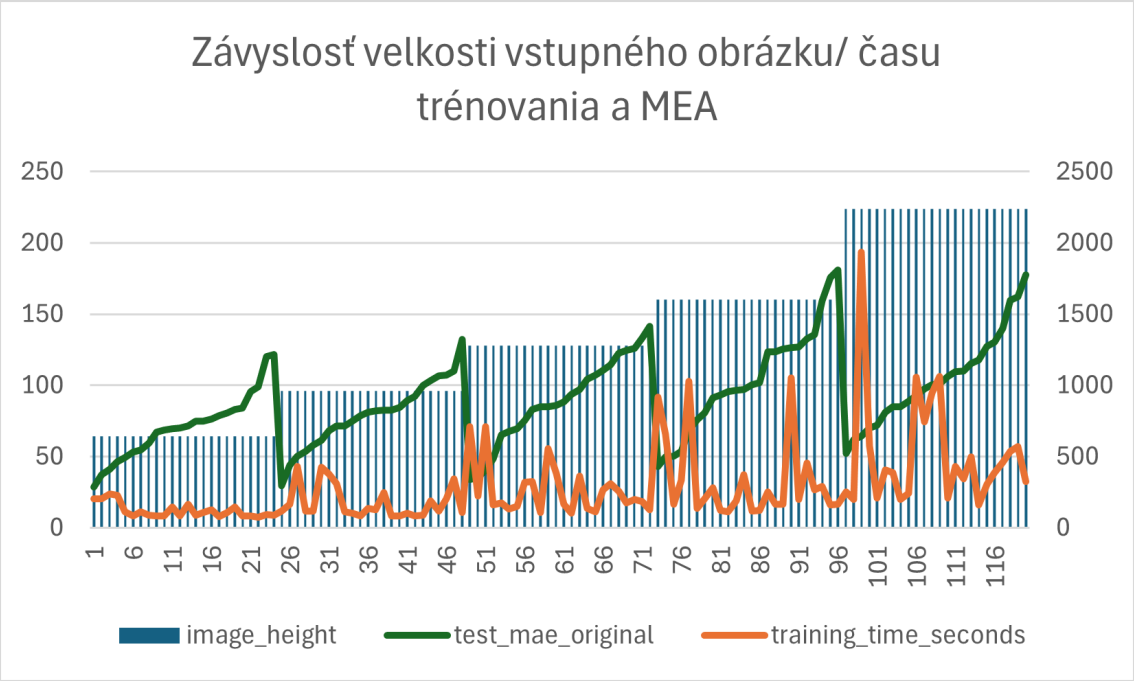
Tento fakt je konzistentný pri všetkých velkostiach sietí ale pri architektúre s označením large je korelácia velkosti obrázkov a času záporná. **??**

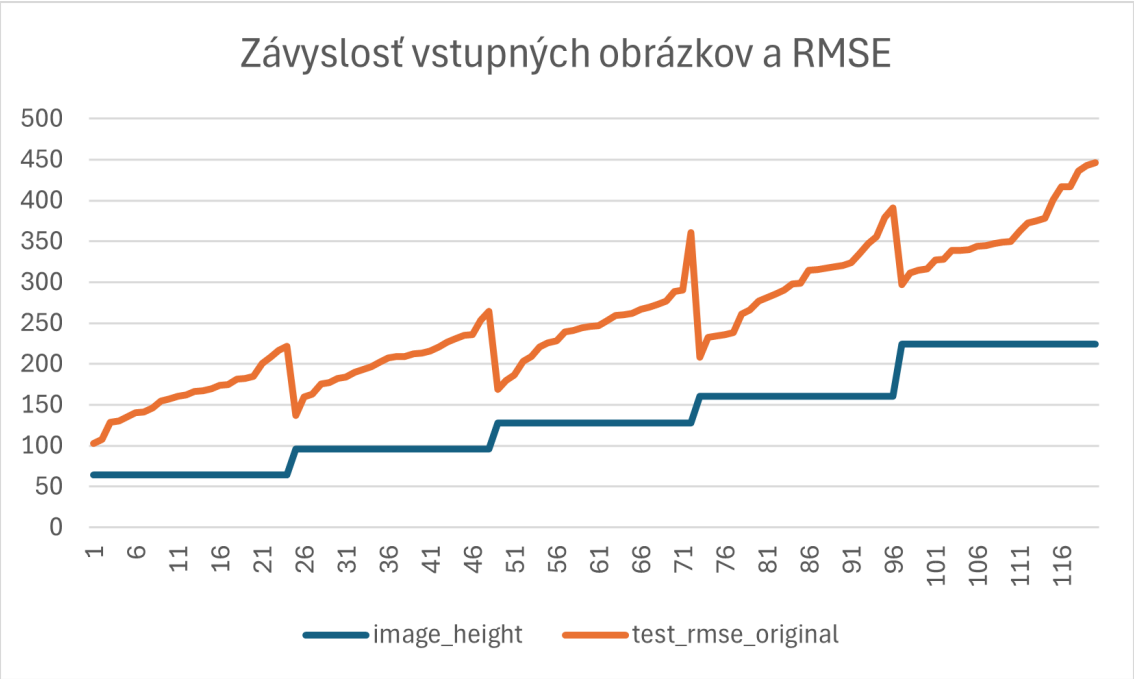
Ako najlepší model z našich testov vyšiel Model Large s najmenšou velkostou vstupných obrázkov 64x64, learning rate 0.001, dropout rate 0.2, dense unit 128 natrénovaný na 42 epochách za 203.4643 sekúnd, priemerná doba trénovania je 282.09974s a medián doby trénovania je 190.6278s. Priemerná obsolútna odchylka (MAE) tothoto modelu je 28.3954 zatial čo medián pre túto hodnotu je 85.0767.

Z obrázku si môžeme všymnuť že zo zväčujúcim sa rozlíšením vstupného obrázu narastá RMSE, z čoho môžeme usúdiť že vstupné dáta ktpré boly zmenšené na menšie rozmery sú lepším vstupom ako dáta v plnom albo väčšom rozlíšený a to bez ohladu na použitú archytektúru alebo iné parametre.

LARGE	čas trénovania	epoch trained	MSE	RMSE	MEA
velkosť obrázkov	-0,3020	0,3614	0,5663	0,5513	0,5531
learning rate	0,2041	0,3828	-0,2729	-0,3534	-0,3995
dropout rate	0,1606	0,2167	0,1234	0,1037	0,1411
danse units	0,1705	0,1842	-0,1889	-0,1604	-0,1662

Tabuľka 5: tabuĺka: Tabuľka závyslostí archytektúry Large





Záver

Cieľom tejto semestrálnej práce bolo oboznámiť sa so základnými princípmi konvolučných neurónových sietí a implementovať model určený na spracovanie obrazových dát a určenie optimálnych rozmerov vstupných obrázkov, ktoré boli vyhodnotené na základe príslušných metrík. V rámci práce boli teoreticky popísané základné časti architektúry konvolučných neurónových sietí, ako sú konvolučné vrstvy, pooling vrstvy, aktivačné funkcie a regularizačné techniky.

V implementačnej časti bola realizovaná implementácia modelu pomocou frameworku TensorFlow a rozhrania Keras. Navrhnutá architektúra obsahovala konvolučné vrstvy, pooling vrstvy, normalizačné mechanizmy a plne prepojené vrstvy určené na spracovanie vstupných obrazových dát.

Súčasťou práce bolo aj vyhodnotenie rôznych rozmerov vstupných obrázkov a ich vplyvu na výslednú presnosť modelu a čas tréningu neurónovej siete. Na základe realizovaných experimentov s rôznymi hyperparametrami konvolučnej siete bolo možné pozorovať vplyv veľkosti vstupných dát na efektivitu procesu učenia a kvalitu výsledného modelu.